



07. Scheduling: Introduction

Workload assumptions

Scheduling Metrics

1. Turnaround time
2. Fairness

FIFO Algorithm

왜 FIFO에 문제가 있는지 - Convoy Effect

Convoy Effect

SJF Algorithm

같은 시간에 도착하지 않고, 어느 때에나 도착한다고 가정해보자

정리

Non-Preemptive (비선점형)

Preemptive (선점형)

STCF Algorithm

New Scheduling Metric

3. Response time

Round Robin Scheduling Algorithm

기존의 SJF

RR: time-slice of 1sec

Time Slice가 짧다면

- 1. Better response time**
- 2. The cost of context switching will dominate overall performance**

Time Slice가 길다면

- 1. Amortize the cost of switching**
- 2. Worse response time**

추가 정보- velog.io/@junttang.

Incorporating I/O

Workload assumptions

- Each job runs for the **same amount of time**. (각각의 Process는 동일한 시간동안 수행된다)
- All jobs **arrive at the same time**. (모든 Process는 같은 시간에 도달한다)

3. All jobs only use the **CPU**. They perform no I/O. (모든 Process는 CPU만 사용한다. I/O가 없음)
4. The **run-time of each job is known**. (각각의 Process의 수행 시간은 미리 알 수 있다)

Scheduling Metrics

1. Turnaround time

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

- 'Process가 수행을 마친 시각' - 'Process가 System에 도착한 시각'
- Turnaround에는 Ready 상태의 시간과 실제 CPU를 사용하는 시간이 포함되어 있어

$$T_{\text{waiting}} = T_{\text{turnaround}} - T_{\text{running}}$$

추가 정보) waiting time: process가 'Ready Queue 안에서 기다리는 시간'

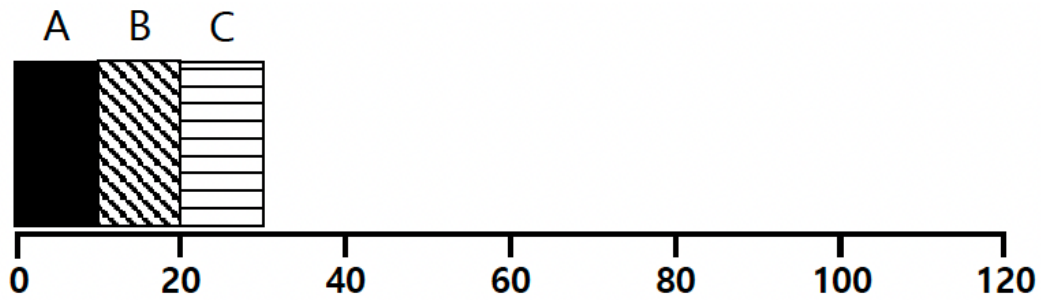
그니까 $T_{\text{arrival}} + T_{\text{waiting}} + T_{\text{running}} = T_{\text{completion}}$ 인듯

2. Fairness

- 모든 Process가 공평하게 CPU를 점유하는지
- Performance와 Fairness는 종종 Trade-Off 관계에 있음

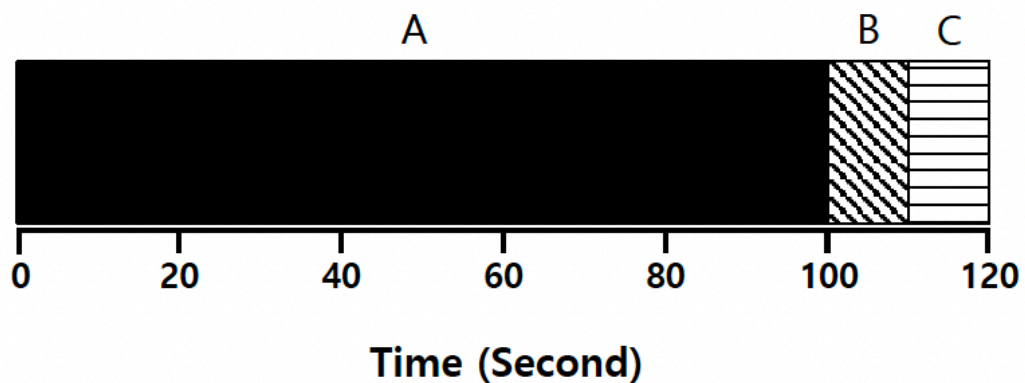
FIFO Algorithm

- First Come, First Served (FCFS)
- 가정) A, B, C 순서대로 도착
- A는 0 ~ 10, B는 0 ~ 20, C는 0 ~ 30 동안 머물렀음
- Average turnaround time = $(10 + 20 + 30) / 3 = 20 \text{ sec}$



왜 FIFO에 문제가 있는지 - Convoy Effect

- 모든 Process의 수행 시간이 같다'는 가정을 없애보자!! - Each job **no longer** runs for the same amount of time.
- 가정) A, B, C 순서대로 도착
- A는 100초 동안, B와 C는 10초 동안



- Average turnaround time = $(100 + 110 + 120) / 3 = 110$
- 또 한편, B, C, A 순서대로 도착한다면?
 - 10, 20, 120이겠네? → 50
- 즉, Arrival 순서와 Process의 수행 시간에 따라 Performance가 상당히 오르락 내리락하는, 불안정한 모습을 보일 것.

Convoy Effect

Ready → Running → Blocked

- 상황 설정

- Ready Queue에 CPU 집약적인 프로세스: 1개 (실행 시간이 긴 놈) - large CPU burst time
- I/O 바운드 프로세스: n-1개 (실행 시간은 짧지만 I/O 작업을 자주 하는 놈들) - less burst time
- 문제 발생 과정
 1. **I/O 바운드 프로세스들이 먼저 CPU 할당받음**
 - 애네는 CPU를 짧게만 쓰고 금방 I/O queue로 이동
 2. **CPU 집약적 프로세스가 CPU 할당받음**
 - 애는 실행 시간이 길어서 CPU를 오래 점유
 - **문제 발생!**
 - I/O 바운드 프로세스들이 I/O 작업 끝내고 Ready Queue로 돌아왔는데, CPU 집약적 프로세스가 아직 CPU 쓰고 있어서 기다려야 함
 - 결과적으로 **I/O 장치들이 놀고 있게 됨 (Idle 상태)**
 - **더 심각한 문제:**
 - CPU 집약적 프로세스가 끝나면 I/O 큐로 이동
 - I/O 바운드 프로세스들은 CPU 받아서 다시 I/O 큐로 이동
 - 근데 CPU 집약적 프로세스가 I/O 장치 점유하고 있어서 기다려야 함
 - 이번엔 **CPU가 놀고 있게 됨**

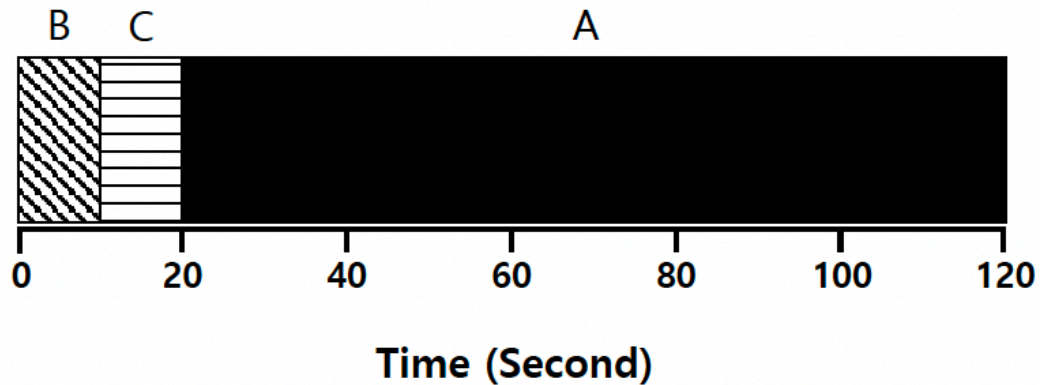


CPU, I/O Device가 모두 Busy한 상황이 가장 이상적이다.
 하지만, Convey Effect가 발생하면 하나의 디바이스 (CPU 또는 I/O Device)만 Busy하고
 다른 디바이스가 Idle한 상태가 발생한다.

SJF Algorithm

- **Shortest Job First Scheduling Algorithm**
- 이름 그대로, 수행 시간이 짧은 Process부터 먼저 돌림

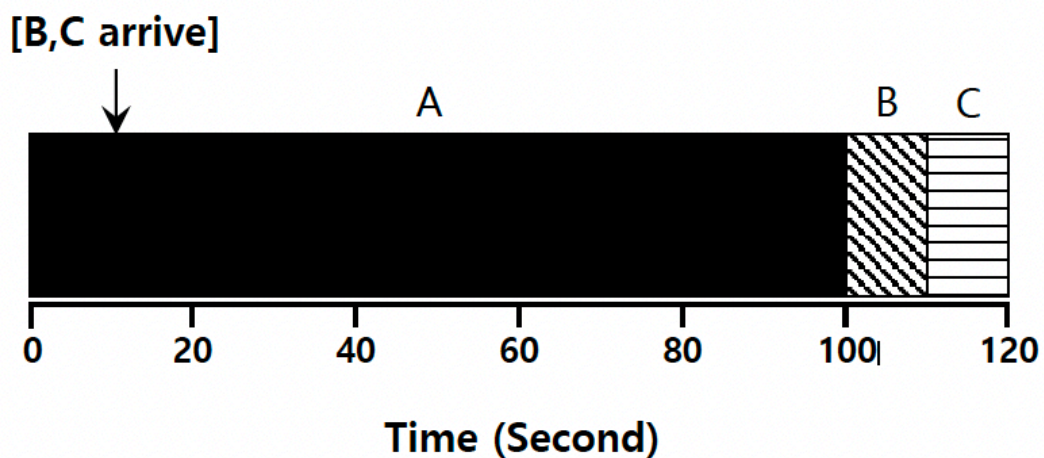
- Non-preemptive scheduler (앞의 녀석 수행이 다 끝나야 다음 Process가 돌아간다)
- 가정) A, B, C 동시 도착
- A는 100초, B와 C는 10초 동안
- Average turnaround time = $(10 + 20 + 120) / 3 = 50$



시험에서 GANT chart 그리라면 이거 그리면 됨

같은 시간에 도착하지 않고, 어느 때에나 도착한다고 가정해보자

- A, B, C 중 A가 먼저 도달하고, 잠시 뒤에 B와 C가 동시에 들어갔다고 해보자
- A 도착: 0 | B, C 도착: 10
- Average turnaround time = $(100 + (110 - 10) + (120 - 10)) / 3 = 103.33$



정리

Non-Preemptive (비선점형)

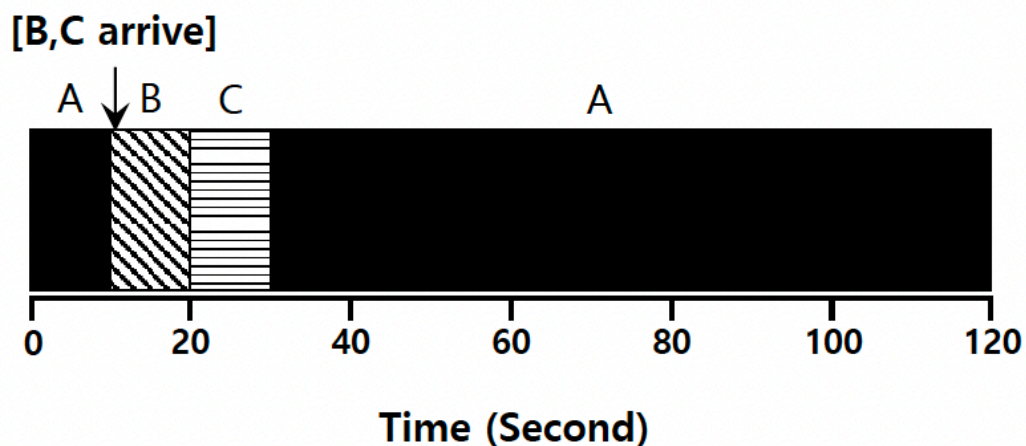
- 한 번 CPU를 할당받으면 **자기가 끝날 때까지** 아무도 못 뺏어감
- 마치 화장실에 들어가면 끝날 때까지 아무도 못 들어오는 것처럼!
- FIFO랑 SJF는 둘 다 Non-Preemptive

Preemptive (선점형)

- CPU 쓰고 있어도 더 우선순위 높은 놈이 오면 뺏김
- 마치 급한 사람이 오면 화장실을 양보해야 하는 것처럼!
- STCF는 Preemptive: 현재 실행 중인 프로세스보다 남은 실행 시간이 더 짧은 프로세스가 도착하면 즉시 교체

STCF Algorithm

- SJF에 preemption을 더한 것
 - Preemptive Shortest Job First (PSJF)
- Process 수행 중간에 남은 부분을 뱉두고 suspend시킨 다음, new job과 비교하여 가장 적은 시간이 남은 job 수행
- A, B, C 중 A가 먼저 도달하고, 잠시 뒤에 B와 C가 동시에 들어갔다고 해보자
- A는 100 동안 수행, B와 C는 10 동안 수행
- A 도착: 0 | B, C 도착: 10



- Average turnaround time = $((120 - 0) + (20 - 10) + (30 - 10)) / 3 = 50$

New Scheduling Metric

3. Response time

$$T_{response} = T_{firstrun} - T_{arrival}$$

- 도착하고 나서 첫번째 응답을 받을 때까지 걸리는 시간
- Job이 System(Ready Queue)에 도착하고 나서부터 처음으로 Schedule될 때까지의 시간
- 앞서 살펴본 FIFO, SJF, STCF등등은 response time의 측면에서 sensitive하지 않다 (좋지 않다)
 - Turnaround Time 측면에선 우수함.
 - 계속 기다려야한다는 근본적인 문제점이 있기 때문에 Response Time이 좋지 않음.

Round Robin Scheduling Algorithm

- 모든 Process에게 'Time Slice' 또는 'Scheduling Quantum'이라 불리는 CPU Time의 작은 단위를 부여한다.
 - 이 time slice는 timer-interrupt period(주기)의 배수이어야 함
 - OS는 Timer Interrupt가 발생해야만 스케줄링 결정을 할 수 있어
 - Timer Interrupt Period가 1ms라면, OS는 1ms, 2ms, 3ms... 단위로만 제어 가능

Schedule: Ready State에서 Running State로 변하는 과정

Preemption: Running State에서 Ready State로 변하는 것

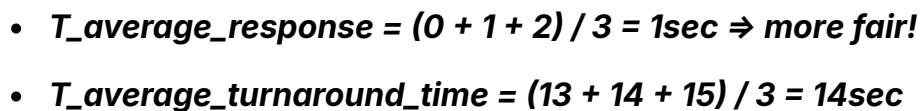
- 이 바로 Schedule

turnaround time과 fairness는 서로 상충관계? 반비례?

기존의 SJF



- ## RR: time-slice of 1sec



Time Slice가 짧다면

1. Better response time

- 8

2. The cost of context switching will dominate overall performance

- (컨텍스트 스위칭 비용이 전체 성능을 지배)
- 프로세스 바꿀 때마다 엄청난 작업이 필요해 (레지스터 저장/복원 등)
- Time Slice가 너무 짧으면 → 실제 일하는 시간보다 바꾸는 시간이 더 많아져!
- 예: 1ms 일하고 5ms 동안 바꾸기... 이러면 비효율적이지?

Time Slice가 길다면

1. Amortize the cost of switching

- (스위칭 비용을 상쇄)
- "Amortize"는 "비용을 나누어 부담한다"라는 뜻
- 길게 실행하면 스위칭 비용이 상대적으로 작아져
- 예: 100ms 일하고 5ms 바꾸기 → 스위칭 비용이 전체의 5%만 차지

2. Worse response time

- (응답 시간이 나빠진다)
- 내 차례가 오려면 오래 기다려야 해
- 예: 10초 Time Slice → 최악의 경우 10초 기다려야 함
- 마치 은행에서 줄이 하나뿐이라 앞사람이 복잡한 업무 보는 동안 계속 기다리는 거야

추가 정보- velog.io/@junttang

- Ready Queue에 n개의 Process, Time Quantum은 q
 - 각 Process는 최대 q라는 Time Slice만큼만 CPU를 점유
 - 따라서 자신을 제외한 n-1개의 Process가 Time Quantum을 Full로 사용한다고 가정하면
 - Process가 기다릴 수 있는 최대 시간은 $(n-1) \times q$
- Time Quantum이 너무 크면 사실상 FCFS Algorithm처럼 작동 - FIFO
 - q가 커질수록 Turnaround Time이 좋아진다(줄어든다)

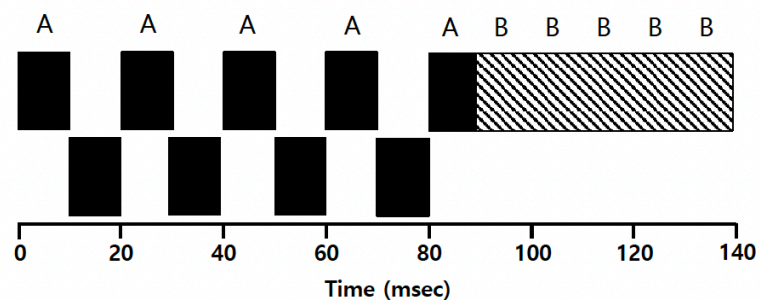
- 꼭 그렇지만은 않을 수 있음
- Time Quantum이 너무 작으면 오히려 그만의 문제



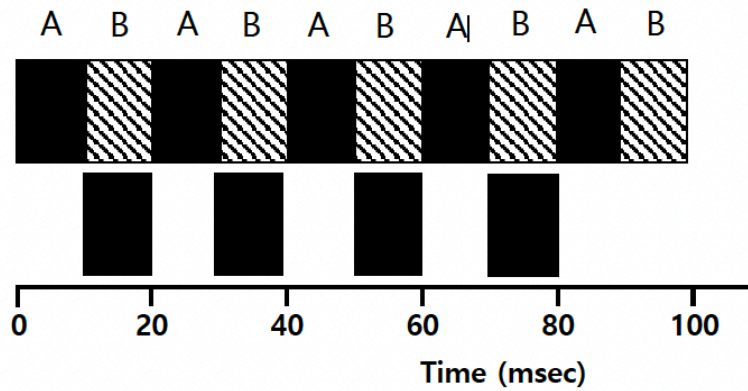
Time Slice의 길이를 어떻게 정하느냐는 Response Time과 Switch Cost 간의 Trade-Off 관계와 관련

Incorporating I/O

- 이전까지 모든 Process가 CPU만 사용한다고 가정.
- 허나, 대부분의 Process는 오히려 I/O Bound인 경우가 많음
- 예시
 - Process A와 Process B가 있음, CPU는 둘 다 50ms 소모
 - A는 10ms CPU하고 10ms I/O
 - B는 단순히 CPU를 50ms 동안 사용하고 no I/O
 - A 먼저 수행하고 B 수행
 - 나쁜 사용



- 좋은 사용



- 특정 Process가 I/O Request를 요청하면,
 - 그 Job는 I/O Operation이 끝날 때까지 Block
 - OS Scheduler는 다른 Process를 Schedule하여 CPU가 놀지 않도록 함
- I/O Operation이 끝나면,
 - Interrupt Exception이 발생
 - OS는 그 Process를 Blocked State에서 Ready State로 바꿔준다. 즉, Ready Queue에 삽입